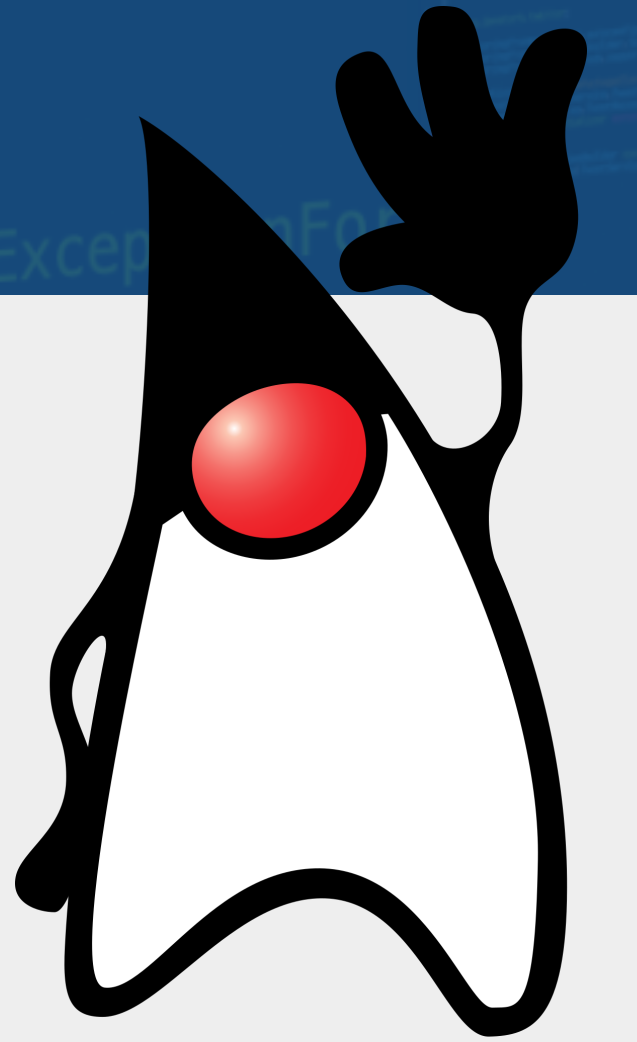


Java Programlama Dili

6. Bölüm

İlkel Tipler ve Değişkenler

Akın Kaldıroğlu



Amaç



- Bu bölümde ilkel tipler (primitive types) ve değişkenler ele alınacaktır
- İlkel tipler arasındaki çevirimler de bu bölümde işlenecektir

Konular



- İlkel Tipler
- Mantıksal Tip: **boolean**
- Karakter Tipi: **char**
- Tam Sayılar
- Gerçek Sayılar
- Sayıların Gösterimi
- **Tip Çevirimleri**
 - Genişleten Çevirimler
 - Daraltan Çevirimler
- **Ödevler**
- **Bundan Sonra?**

İlkel Tipler



İlkel Tipler - I



- Daha önce görmüştük, ilkel tipler (primitive types, primitives) Java'ya aritmetik, karakter ve mantıksal işlemleri kolayca yapabilmek için konmuştur
- İlkel tipler dilin tasarımı sırasında detayları değişmeyecek şekilde belirlenmiştir ve yenileri de oluşturulamaz
- Karmaşık ya da referans tiplerin aksine ilkel tiplerin sınıfı yoktur dolayısıyla sınıftan türetilmiş nesne de değildirler

İlkel Tipler - II



- İlkel tipler (primitive types ya da primitives) 8 tanedir:
 - **byte, short, int, long, float, double, char** ve **boolean**
 - Sayısal olmayan (non-numeric) tek tip, mantıksal (logical) tiptir:
boolean
- Geriye kalan 7 tip de sayısaldır (numeric)
 - **byte, short, int, long, float, double**, doğrudan sayıları temsil eder

İlkel Tipler - III



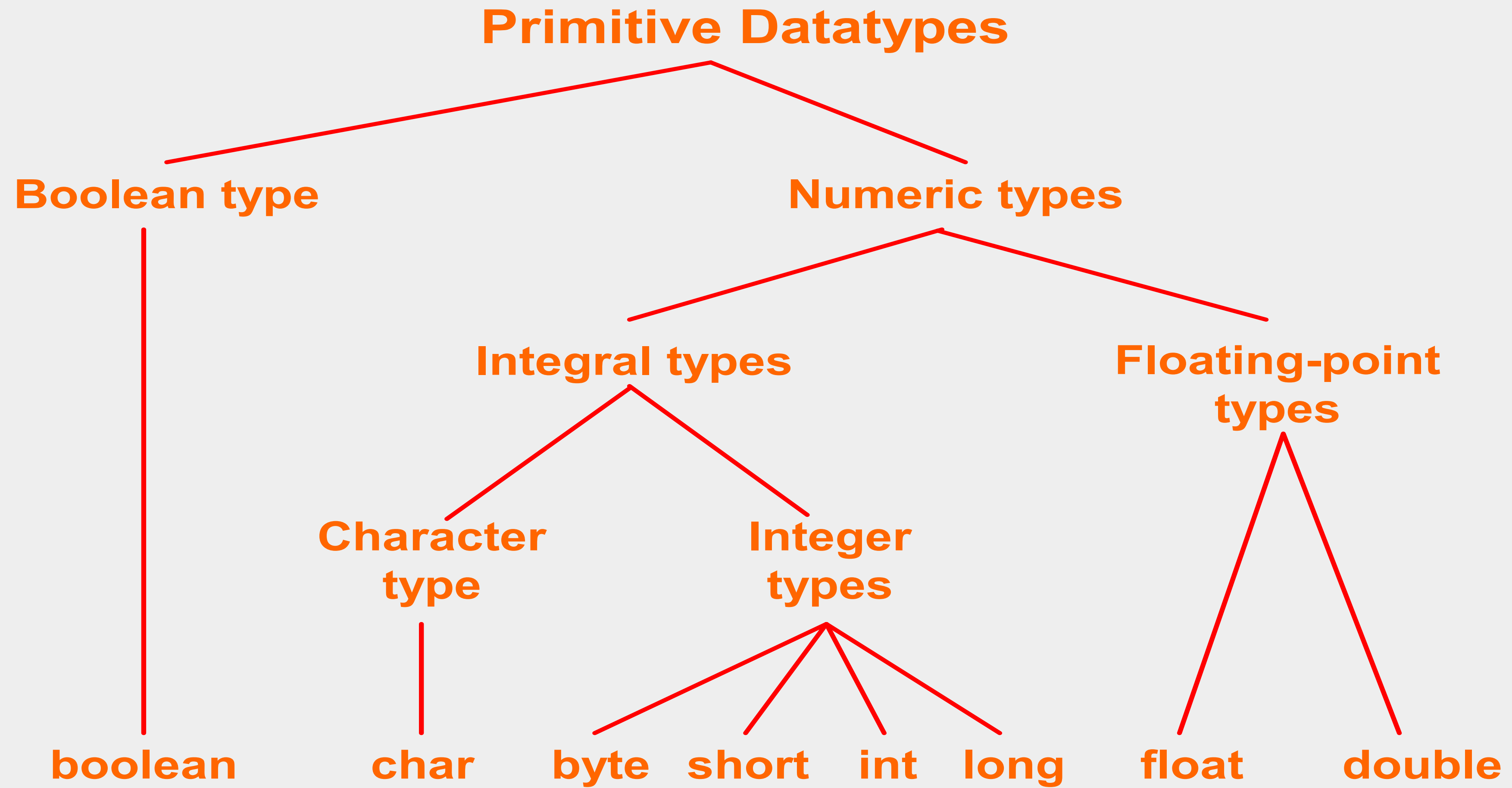
- **char**, karakter tipidir ve her karakterin sıfır ya da bir pozitif tam sayı değeri vardır

İlkel Tipler - IV



- **char** dışında diğer bütün 6 sayısal tip işaretlidir (signed), pozitif ve negatif değerler alabilir
- Java'da unsigned (işaretsiz) sayı tipi yoktur
- 6 sayısal tipin 4'ü tam sayılar, 2'si ise gerçek (real) sayılar içindir:
 - Tam sayı tipleri: **byte**, **short**, **int** ve **long**
 - Gerçek sayı tipleri: **float** ve **double**

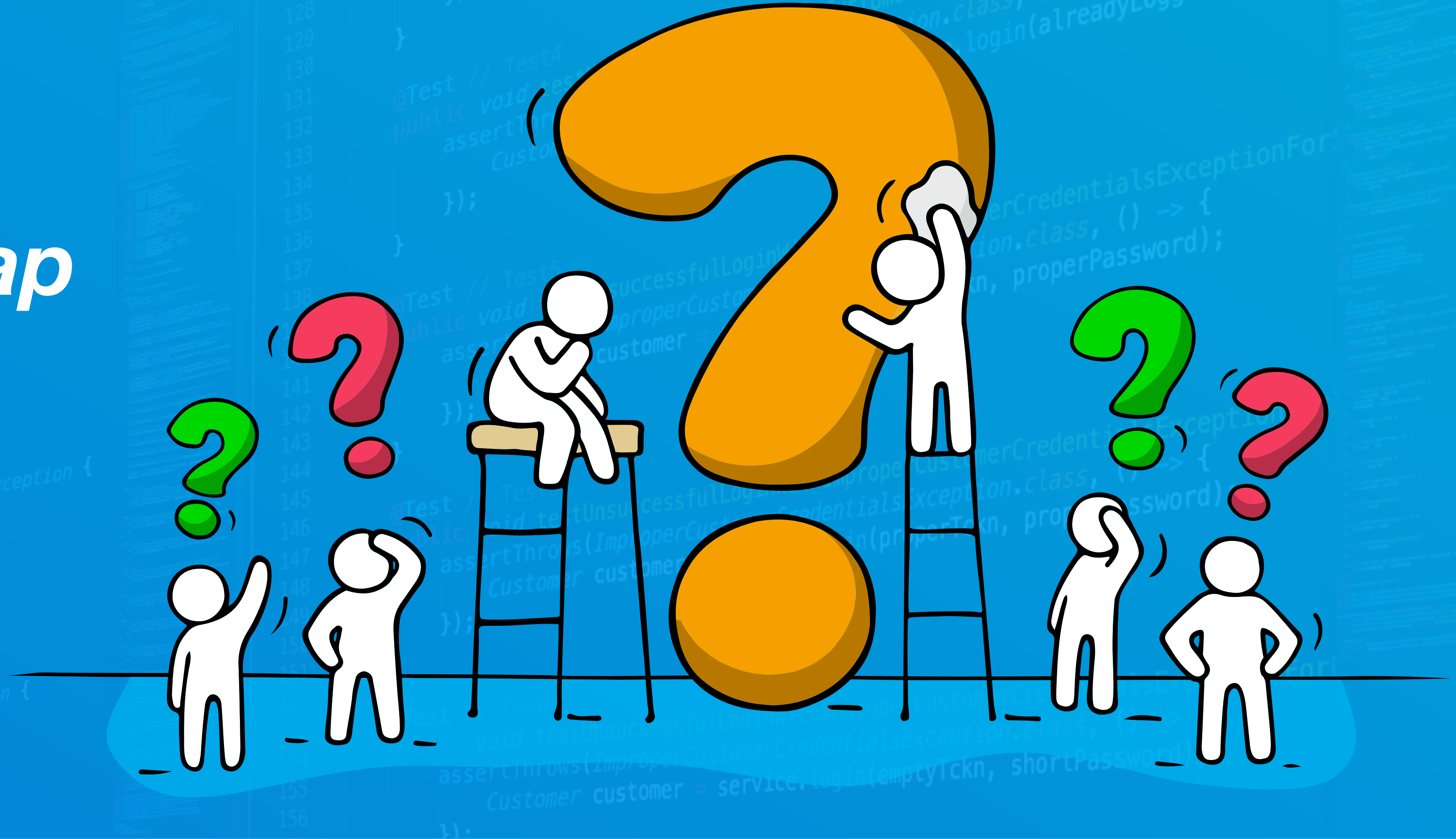
İlkel Tipler - IV





- *Karmaşık ya da referans tiplerin aksine ilkel tiplerin sınıfı yoktur dolayısıyla sınıftan türetilmiş nesne de değildirler* dedik
- İlkel tiplerin nesne hallerini ifade etmek üzere oluşturulmuş, **Integer** ya da **Character** gibi *sarmalayan/wrapper* sınıflar vardır, ileride ele alacağız
- Ayrıca ilk anda akla gelenin aksine **String** bir ilkel tip değildir, pek çok karakterden oluşan ve pek çok metoda sahip bir referans tiptir, bir sınıftır

Soru ve Cevap Zamanı!



Mantıksal Tip: boolean



boolean



- **boolean** ilkel mantıksal (logical) tiptir, doğru ya da yanlış olan bir doğruluk durumunu ifade etmek için kullanılır
- Değişkenleri sadece **true** ya da **false** değerlerini alabilir:

```
boolean flag = false;  
boolean isClosed = true;  
boolean isCreated;
```

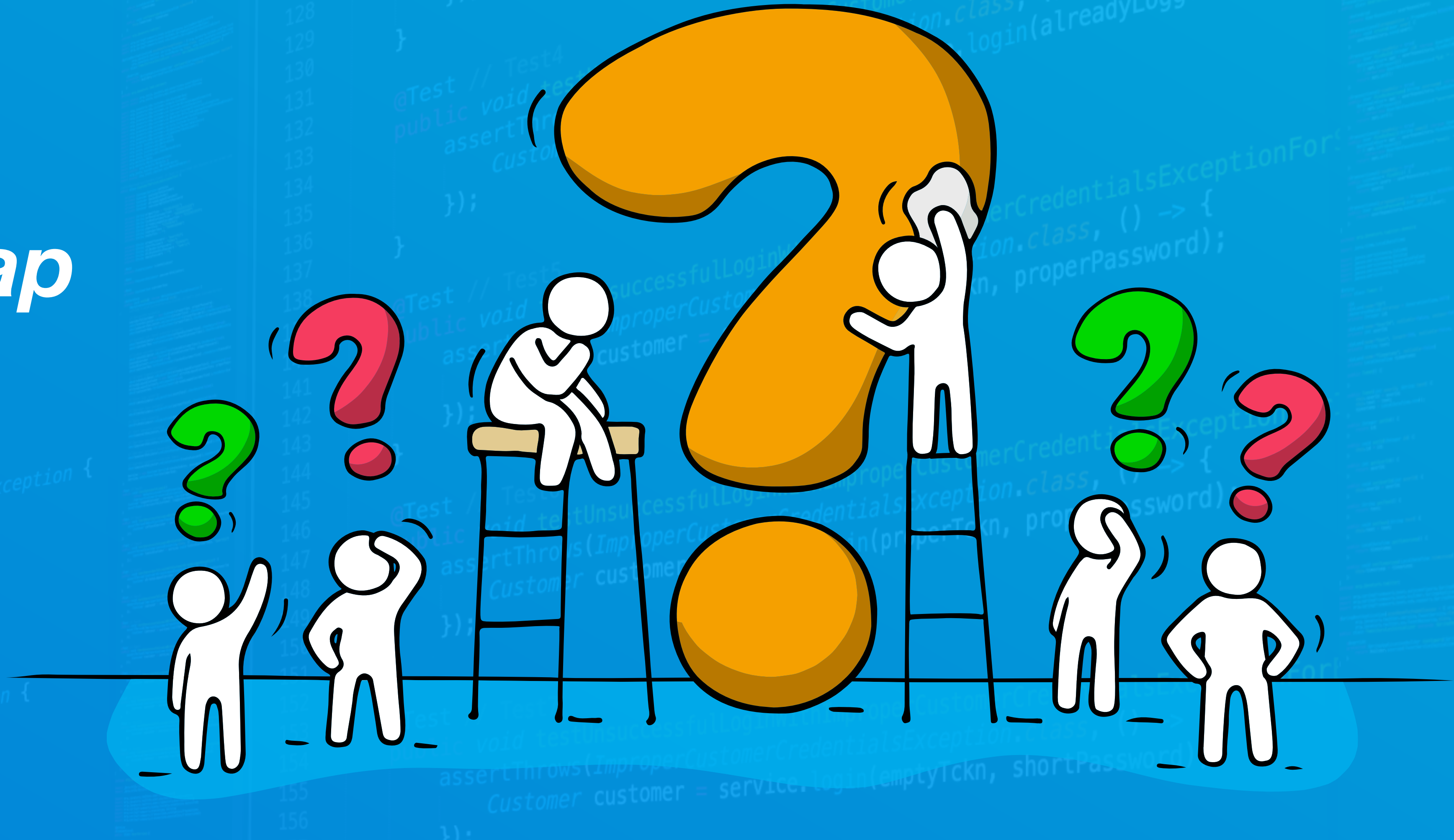
- Bazı dillerde var olan, sayıların mantıksal değer ifade etmeleri (**true** için 1, **false** için 0 örneğin) Java için geçerli değildir

Booleans



- `Booleans`

Soru ve Cevap Zamanı!



Karakter Tipi: char



char - I



- Karakter tipi **char** ile ifade edilir
- Bir karakter sabitesi, iki tane tekli kesme işareti ' ' arasında ifade edilir

```
char firstLetter = 'a';  
char c;
```

- Java **Unicode** karakter setini kullanır,
- Unicode, Türkçe ve pek çok dünya dilinin karakterlerini içeren bir standarttır

char - II



- Tarihi olarak karakterlere sayısal değerler verilmiştir
- Örneğin ASCII karakter seti 7 bitliktir ve karakterlerin 0 ile 127 arasında sayısal değerleri vardır
- Unicode karakterleri 16 bitliktir dolayısıyla karakterler 0 ile $2^{16}-1 = 65,535$ arasında değerler alır
- Unicode'un ilk 128 karakteri ASCII karakterlerinden oluşur

'a'	⇒	97
'A'	⇒	65
'9'	⇒	57
' '	⇒	32

char - III



- **char** negatif değer alamaz, sıfır ya da pozitif değer alabilir
- Bir **char** değişkene negatif değer atamak derleme hatasıdır, negatif değer ancak ileride göreceğimiz gibi çevirim işlemcisi (cast operator) ile atanabilir
- Çevirim işlemcisi ile negatif değer atanırsa gerçek değer, 65,536'nın gerektiği kadar katının eklenmesiyle 0 - 65,535 arasında belirlenir

```
char c = (char) -65471 // A
```

Characters



- `characters.Characters`

Unicode - I



- Unicode (unicode.org), 16 bitlik yani 65,536 (64K) karakterli geniş bir settir
- Unicode karakterleri `\uXXXX` şeklinde ifade edilirler; buna **UTF-16** denir
 - **x**, 16'lık sistemdeki (hexadecimal) rakamlardır
 - Dolayısıyla bir Unicode kodu, 4 tane 4 bitlik rakamdan oluşur
 - **char**'ın alabileceği değerler `\u0000` ile `\uffff` (0 to 65535) arasındadır

Unicode - II



- Unicode karakter setinin ilk 128 karakteri 7-bit **ASCII** (<https://www.asciitable.com/>), ilk 256 karakteri ise 8-bit **ISO 8859-1 (ISO Latin-1)** setini oluşturur
- Klavyenizde olmayan karakterler için Unicode karşılıklarını kullanabilirsiniz
- Bu kodlar birden fazla karakterden oluşsa bile tek karakter olarak ('\u00a7') yazılırlar

' '	\u0020	Boşluk (Space)
'0'	\u0030	0
'9'	\u0039	9
'A'	\u0041	A
'Z'	\u005a	Z
'ç'	\u00e7	ç
'ß'	\u00a7	ß

Kaynak Kod Karakter Kodu - I



- Java karakterleri Unicode olduğundan programlarda Türkçe, Japonca, Arapça, İbranice vs. karakterler kullanabilirsiniz
- Derlerken "`-encoding xxx`" ile "`xxx`"i kaynak dosyasında kullanılan karakter kodu (encoding) olarak gösterebilirsiniz
 - Japonca için *EUC-JP*, Türkçe için *ISO8859_9* ya da *UTF-8* kullanılabilir.
 - Eğer bu ayar (flag) kullanılmazsa platformun karakter kodu kullanılır

Kaynak Kod Karakter Kodu - II



- IDE'ler de varsayılan halde platformun karakter kodunu kullanırlar ve tüm IDE ya da tek tek projeler için karakter kodunun belirlenmesine izin verirler
- IntelliJ'de `Settings => Editor => File Encodings` sayfasında `Project Encoding`'in değerini değiştirebilir, örneğin *UTF-8* yapabilirsiniz
- Eclipse'de proje üzerinde sağ tık `Properties => Resource` sayfasında `Text File Encoding`'in değerini değiştirebilirsiniz

Kaynak Kod Karakter Kodu - II



- Geniş bilgi için:
- JDK 25 <https://docs.oracle.com/en/java/javase/25/intl/supported-encodings.html>
- JDK 8 <https://docs.oracle.com/javase/8/docs/technotes/guides/intl/encoding.doc.html>



Uygulama



- Aşağıdaki örneklerin doğru derlenmesi ve çalışması için IDE'de projenin Text File Encoding'ini UTF-8 yapın
- `characters.UnicodeSelam`
- `characters.TurkishCharacters`



- Aynı örnekleri komut satırında `-encoding` ile çalıştırın
- Doğru ve yanlış kodlar girerek Java derleyicisinin davranışını gözlemleyin
- Örneğin aşağıdaki yanlış ve doğru karakter kodları ile derleyin:

```
javac -encoding EUC-JP TurkishCharacters.java
```

```
javac -encoding UTF-8 TurkishCharacters.java
```



- Çalıştığınız platform, kullandığınız JDK veya IDE'nin karakter ayarları vs. bilgileri öğrenmek için bu örneği çalıştırabilirsiniz
- Bu örneği hem IDE'nizde hem de komut satırında çalıştırın
- `characters.EnvironmentInfo`

Escape Dizisi



- Bazı özel karakterlere kaçış dizisi (escape sequence) adı verilir ve dilde bunlar için özel karakterler vardır:

<code>\b</code>	<code>\u0008</code>	Backspace
<code>\t</code>	<code>\u0009</code>	Horizontal tabulation
<code>\n</code>	<code>\u000a</code>	Linefeed
<code>\f</code>	<code>\u000c</code>	Form feed
<code>\r</code>	<code>\u000d</code>	Carriage return
<code>\'</code>	<code>\u0027</code>	Apostrophe-quote
<code>\"</code>	<code>\u0022</code>	Quotation mark
<code>\\</code>	<code>\u005c</code>	Backslash

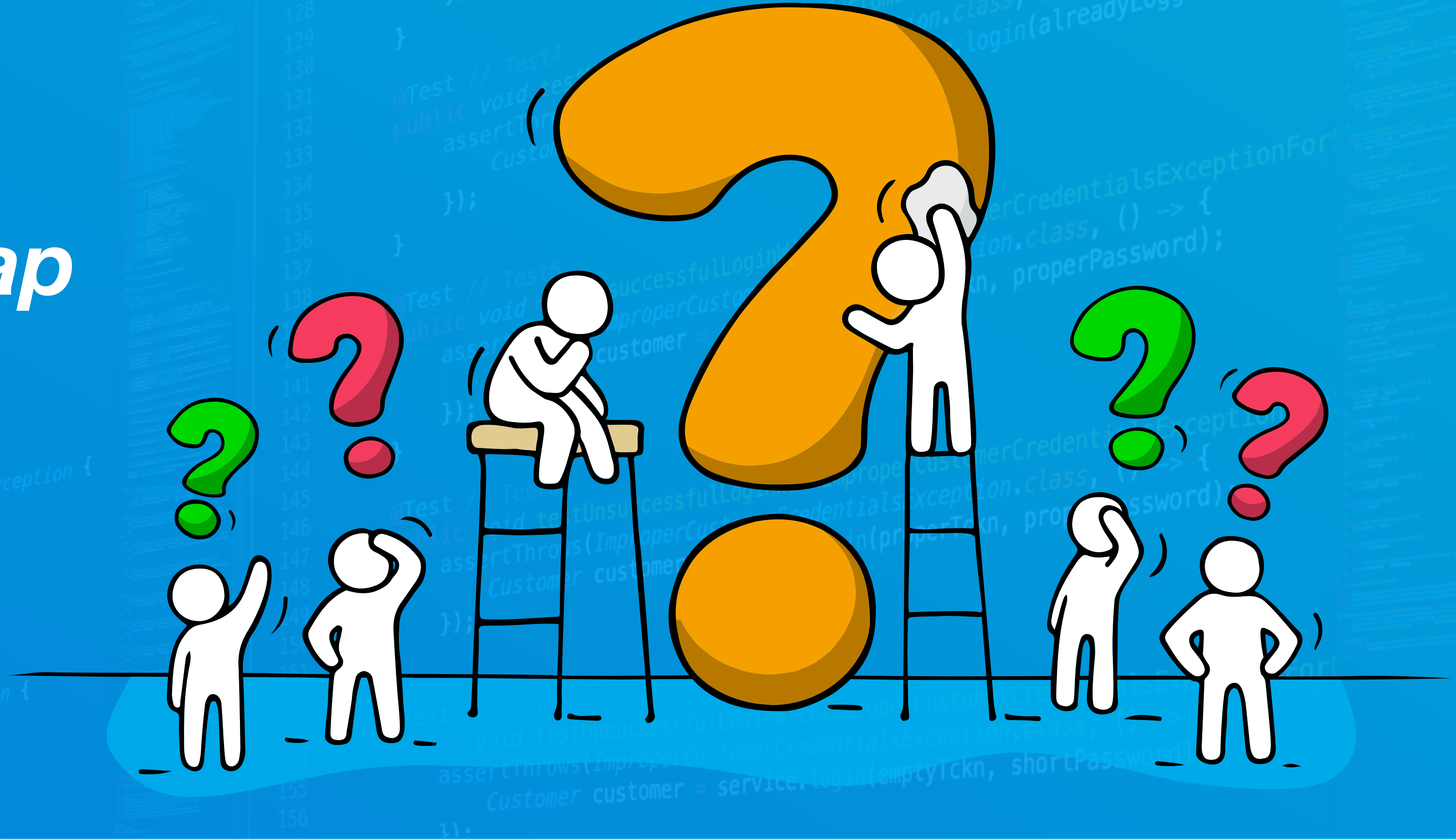
- Bu kodlar birden fazla karakterden oluşsa bile tek karakterlerde olduğu gibi, ```, `'`, iki tane tekli kesme işareti arasında yazılırlar, ``\n'` gibi

Escapes



- `characters.Escapes`

Soru ve Cevap Zamanı!



Tam Sayılar



Tam Sayılar (Integers) - I



- Java'da 4 tane tam sayı (integer) tipi vardır: `byte`, `short`, `int` ve `long`
- Bit cinsinden uzunlukları ve alabilecekleri değerlerin aralıkları (range) şöyledir:
- Bütün sınır değerler de aralığa dahildir

İsim	Uzunluk	Alt Sınır	Üst Sınır
byte	8	-128	127
short	16	-32.768	32.767
int	32	-2.147.483.648	2.147.483.647
long	64	-9.223.372.036.854.775.808	9.223.372.036.854.775.807

Tam Sayılar (Integers) - II



- Tam sayı tiplerde varsayılan tip **int**'dir
- Kodda tipi belirtilmeden 5 gibi bir tam sayı kullanıldığında bu bellekte **32 bit** olarak temsil edilir
- Bir tam sayıyı **long** olarak belirtmek isterseniz ya da zaten sayı **int**'e sığmayacak kadar büyükse sayının sağında **L** ya da **1** kullanabilirsiniz
- Okuma kolaylığı için **L** tercih edilmelidir

```
int i = -5;  
long l = 47;
```

```
long l = 47L;  
long l = 47L;
```

```
long l = 9355032854873; // Error! Out of range  
long l = 9355032854873L;
```

Tam Sayılar (Integers) - II



- Benzer şekilde **byte** ya da **short** tam sayı belirtmek mümkün değildir

```
byte b = 8;  
short s = -12;
```

- **byte** ya da **short** tiplerden olan değişkenlere yapılan atamalarda, sağdaki değer 8 ya da 16 bite otomatik olarak çevirilir (conversion)
- Tabi olarak sağ taraftaki tam sayı değerinin, 8 ya da 16 bit ile temsil edilebilmesi gereklidir, aksi taktirde sabite atamada derleme hatası alınır, ifadede ise çalışma zamanında garip sonuçlar çıkabilir

Tam Sayılar - IV



- Bir uygulamada bellek tasarrufu için **int** yerine **byte** ya da **short** kullanmayın, astarı yüzünden pahalıya gelebilir
- **byte** daha çok gömülü (embedded) uygulamalarda, aşağı seviye (low-level) I/O arayüzlerinde kullanılır,
- Bu tip yapılarda **byte** dizilerine (**byte []**) de sık rastlanır

Integers



- `numbers.Integers`

İkilik Sistem - II



- İkilik sistemde pozitif sayılar 0, negatif sayılar 1 ile başlar
- Negatif sayılar, aynı sayının pozitifinin ikinin tümleyeni (two's complement) yöntemiyle dönüştürülmesiyle elde edilir

[illegible]

- Bu örnekler 32-bit yani `int` tipi içindir, `long` için bit sayısını 64'e çıkarmak ve soldaki 32 biti 0 ile doldurmak gerekir

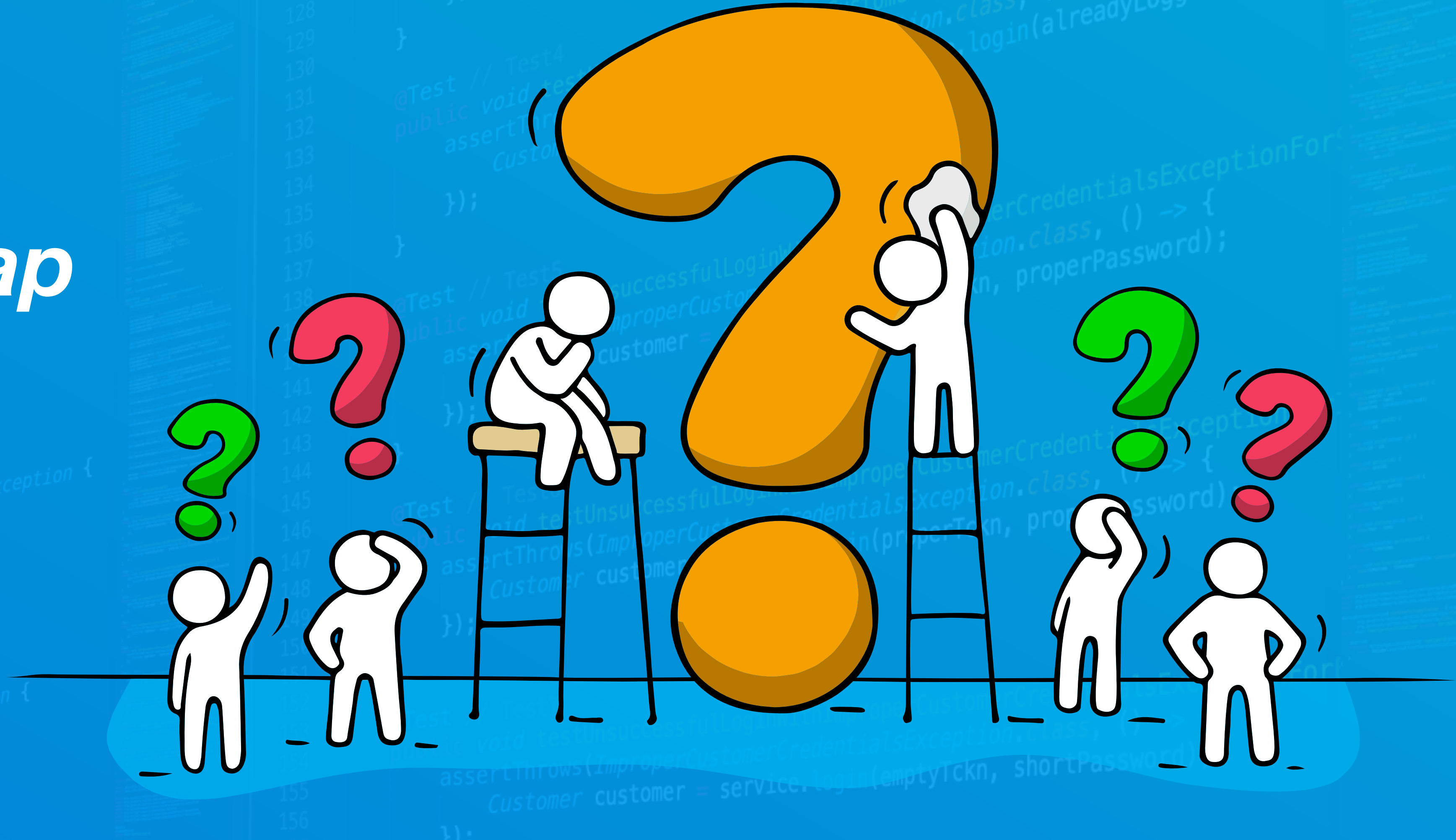
[illegible]

IntegerBinaryConverter



- `numbers.IntegerBinaryConverter`

Soru ve Cevap Zamanı!



Gerçek Sayılar



Gerçek Sayılar (Real Numbers) - I



- Gerçek (reel, gerçel) sayılar (real numbers) şunlardır:
 - Tam sayılar (integers): -5, 0, 2, 7 gibi
 - İki tam sayının oranı olarak (a/b , a ve b tam sayı ve b , 0'dan farklı) yazılabilen kesirli sayılar (rational numbers): $5/2 = 2.5$, $1/4 = 0,25$
 - Kesirli sayılar (fractional numbers), tam sayıları kapsar: $6 = 12/2$
 - Kesirli sayılara ondalıklı sayılar da denir (decimal numbers): $1.2 = 6/5$
 - Ondalıkli sayılarda virgülden sonraki sayılar tekrar edebilir: $1/3 = 0.3333\dots$

Gerçek Sayılar (Real Numbers) - II

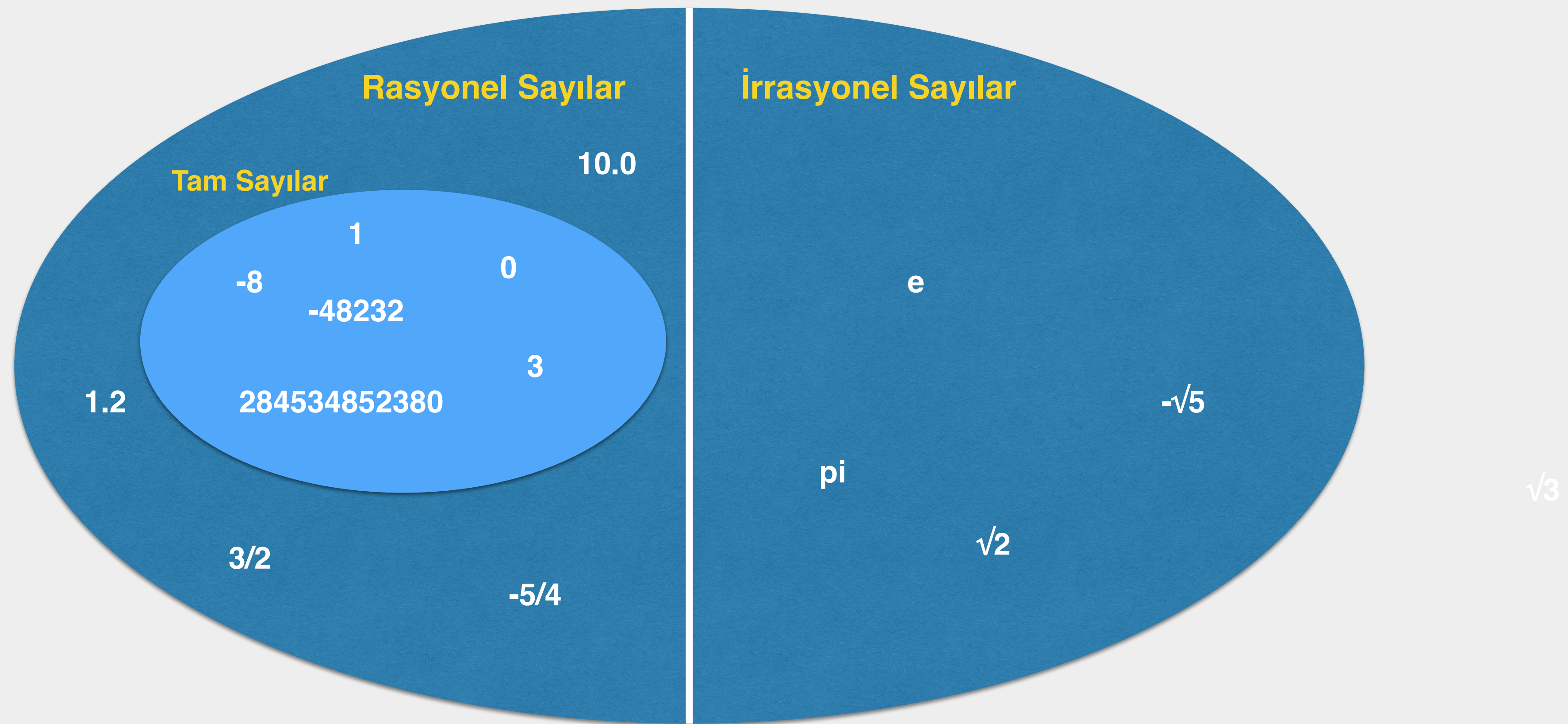


- Kesirli yazılamayan irrasyonel sayılar (irrational numbers),
 - $\sqrt{2}$ (2'nin kökü), $\sqrt{3}$, $\sqrt{5}$, π (pi) ya da e gibi

Gerçek Sayılar (Real Numbers) - II



Gerçek Sayılar (Reel/Gerçel Sayılar)



Kayan Noktalı (Floating-Point) - I



- Gerçek sayılar, bilgisayar aritmetiğinde **floating-point (kayan noktalı)** olarak temsil edilirler
- Batılı ülkelerde sayılarda nokta kullanır ama biz virgöl kullanırız
- Bu gösterim için *IEEE 754 Binary Floating-Point Arithmetic* standardı oluşturulmuştur
- Bu standartta gerçek sayılar, 32 bit olarak tek hassasiyet (single-precision) ve 64 bit olarak çift hassasiyette (double-precision) ifade edilirler

Kayan Noktalı (Floating-Point) - II



- 32 ya da 64 bit ile bir aralıktaki tüm gerçekte sayılar ifade edilemez
- Tam sayılar sayılabilirler (countable) ama gerçekte sayılar sayılamazlar (uncountable), örneğin iki tam sayı arasında sayılabilir sayıda tam sayı varken sayılamaz sayıda gerçekte sayı vardır, dolayısıyla devamlı yuvarlamalar yapılır
- Ayrıca 10'luk sistemde $\frac{1}{3}$ gibi ifadelerin gösteriminde olan sonsuz tekrarlar, 2'lik sistemde de ortaya çıkar, örneğin 0.1'de
- Dolayısıyla kayan noktalı gösterim, gerçekte sayıları ifade etmede bazı kısıtlara sahiptir

Java'da Kayan Noktalı Gösterim



- Java'da iki farklı hassasiyette kayan noktalı gösterim için 2 tip vardır:
 - **float** 32 bittir
 - **double** 64 bittir
- Java'nın **float** ve **double** tipleri *IEEE 754 Binary Floating-Point Arithmetic* standardına göre oluşturulmuştur

HypotenuseCalculator



- `numbers.HypotenuseCalculator`

float ve double - I



- Gerçek sayılarda varsayılan tip **double**'dir, yani kodda tip belirtilmeden 3.14 gibi bir sayı yazılırsa Java derleyicileri bunu 64 bit ile yani **double** olarak gösterirler
- Gerekli olmasa da istenirse sayının yanında **d** ya da **D** kullanılabilir
- Bir gerçek sayıyı 32 bit yani **float** olarak belirtmenin yolu ise sayının yanında **f** ya da **F** kullanmaktır

```
float pi =      3.14f;  
float e =      2.71828F;  
double cap =    1.22;  
double length = 10.0d;  
double alan =   88.15D;
```

PiDemo



- `numbers.PiDemo`

float ve double - II



- **float** ile ifade edilebilecek en küçük sayı $1.40e-45$, en büyük sayı ise $3.4028235e38$ 'tir
- **double** ile ifade edilebilecek en küçük sayı $4.9e-324$, en büyük sayı ise $1.7976931348623157e308$ 'tir

float ve double - III



- **float** ve **double** sayıların ifade edilmesindeki kısıtlar sıklıkla bu sayılarla yapılan işlemlerde hatalara sebep olur

```
0.1f + 0.1f + 0.1f + 0.1f + 0.1f + 0.1f + 0.1f =?  
0.1d * 0.1d =?
```

- Hatta aynı değerin **float** ve **double** temsilleri arasında bile farklar bulunabilir

```
float fPi = 3.14F;  
double dPi = 3.14D;  
fPi == dPi // false
```

FloatingPointDemo



- `numbers.FloatingPointDemo`

float ve double - IV



- Para hesabı gibi hassas işlemlerde bu tür hatalardan kaçınmak için **float** ya da **double** yerine Java'nın `BigDecimal` gibi yapılarını kullanmak gereklidir

İkilik Sistem - II



- İkilik sistemde pozitif sayılar 0, negatif sayılar 1 ile başlar
- Gösterimde de IEEE 754 standardı kullanılır
- Kayan noktalı sayıların ikilik sistemde tam olarak ifade edilememesinin bir başka sebebi de bazı hanelerinin sonsuz tekrarıdır
- Örneğin $1/3 = 0,333333\dots$, onluk sistemde tekrarlayan 3'e sahiptir

```
1.0 = 1.0 // Exact representation
0.625 = 0.1011 // Exact representation
0.1 = 0.00011001100110011001100110011001 // Repeating 1100
1.0/3.0 = 0.01010101010101010101010101010101 // Repeating 0101
```

FPBinaryConverter



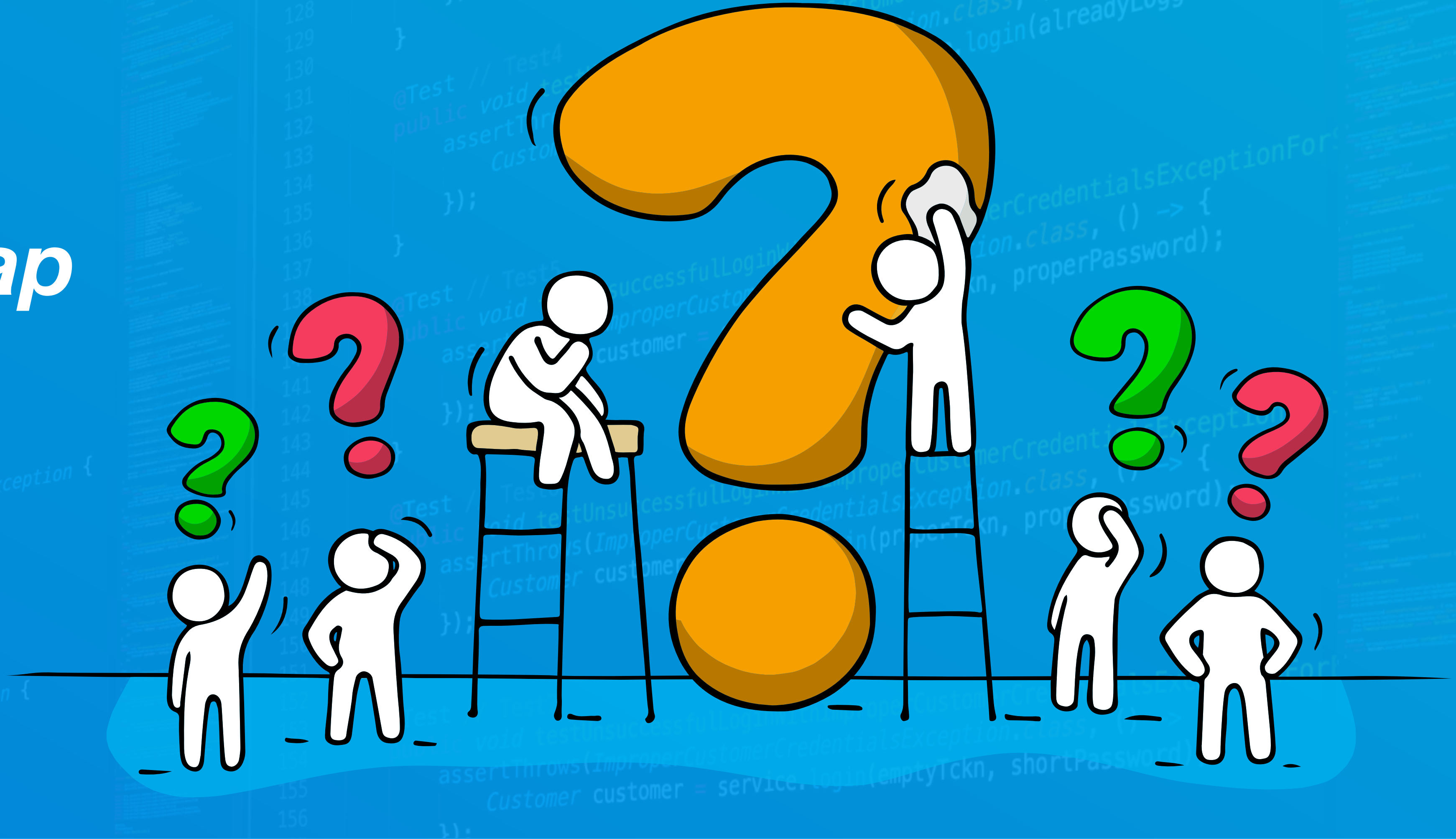
- `numbers.FPBinaryConverter`

IEEE754Demo



- `numbers.IEEE754Demo`

Soru ve Cevap Zamanı!



Sayıların Gösterimi



2'lik, 8'lik ve 16'lık Tam Sayılar - I



- Java'da tam sayılar, (**byte**, **short**, **int**, ve **long** değerleri) **2'lik** (2 tabanlı, binary), **8'lik** (8 tabanlı, octal) ve **16'lık** (16 tabanlı, hexadecimal) olarak ifade edilebilir
- Java'da en başından bu yana 0 ile başlayan sayılar 8'lik, **0x** ya da **0X** ile başlayan sayılar ise 16'lık tabanda işlenirler:

```
0132 = 1*8^2+3*8^1+2*8^0 = 90
```

```
0x5a = 5*16^1+10*16^0 = 90
```

```
0xffff = 15*16^3+15*16^2+15*16^1+15*16^0 = 65535
```

2'lik, 8'lik ve 16'lık Tam Sayılar - II



- Java SE 7 ile birlikte, tam sayıların 2'lik tabanda (binary) ifade edilmesi de mümkün hale gelmiştir
- Bu amaçla gösterimde `0b` ya da `0B` kullanılır

```
0b1010 = 1*2^3+0*2^2+1*2^1+0*2^0 = 10
```

```
0B1111 = 1*2^3+1*2^2+1*2^1+1*2^0 = 15
```

BinaryOctalHexadecimalIntegers



- `numbers.BinaryOctalHexadecimalIntegers`

Üstel İfadeler



- Kayan noktalı sayılar, **üstel (exponential)** olarak da yazılabilir
- Bunun için **e** ya da **E** kullanılır:
- Üstel gösterim sadece **float** veya **double** tipler için geçerlidir, tam sayılar bu şekilde yazılamaz

```
double d1 = 1.6e-19;  
float f1 = 6.02E23F;  
  
int i = 1E1; // Error!
```

Sayısal Değerlerde _ Kullanımı - I



- Java SE 7 ile sayısal ifadelerin okunmasını kolaylaştırmak için _ alt çizgi (underscore) kullanımı gelmiştir:
- _, bütün sayısal sabitelerde, tam, kayan noktalı ve üstel sayılarda, ancak rakamların arasında kullanılabilir,
- 0b/0B, 0 ve 0x/0X ile başlayan 2'lik, 8'lik ve 16'lık tam sayılarla kullanılabilir

```
int i      = 10_000_000;  
double d   = 2_577.930_113;  
float f    = 6.02E2_3F;  
int b      = 0b1010_1010_1010;
```

Sayısal Değerlerde _ Kullanımı - II



- `_` sadece iki rakam (digit) arasında kullanılmalıdır
- Dolayısıyla şu yerlerde `_` kullanılamaz:
 - Başta ve sonda,
 - `.`'nin yanında,
 - `f`, `F`, `d`, `D`, `l` ve `L`'nin yanında

Sayısal Değerlerde _ Kullanımı - II



- _ sadece sayısal sabitelerde kullanılabilir,
- Sayıların metinsel (textual, string) sunumlarında kullanılması problemlidir, sayıya çevrildiğinde hata (**NumberFormatException**) alınır

```
// Problem! Can't convert to int  
String s = "10_000_000";
```

Underscores



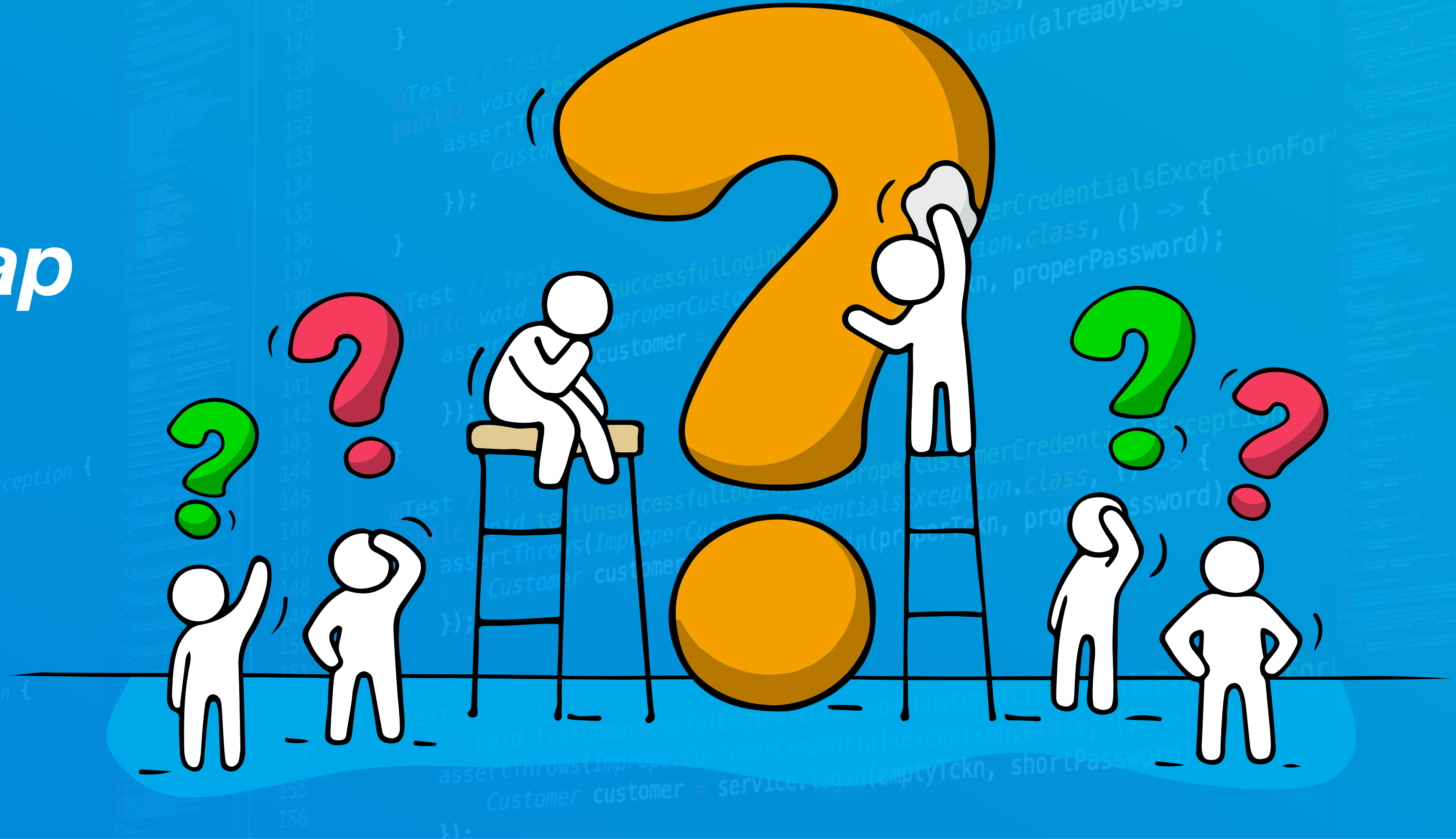
- `numbers.Underscores`

Primitives



- `Primitives`
- Bu örnekteki,
 - Geçerli ve geçersiz değişken isimlerine,
 - Değişkenlere atanabilecek değerlere,
 - Üstel kullanımlara,
 - Ve diğer bazı basit noktalara
- dikkat edin

Soru ve Cevap Zamanı!



Tip Çevirimleri



Tip Çevirimleri - I



- Bir değeri, sahip olduğu tipten farklı tipteki bir değişkene atamaya **çevirim** ya da **dönüştürme (conversion)** denir
- Java, kuvvetli tipli (strongly-typed) bir dil olduğundan, gerek ilkel gerek karmaşık tipler arasındaki çevirimlerle ilgili hem derleme hem de çalışma zamanında kontroller yapar
- Bu kısımda sadece ilkel tipler arasındaki çevirimleri ele alacağız
- Karmaşık tipler arasındaki çevirimleri ileride inceleyeceğiz

Tip Çevirimleri - II



- İlkel tipler arasında üç türlü çevirim vardır:
- İmkansız çevirim (**impossible conversion**)
- Bit yapısı olarak dar olandan geniş olana: **Genişleten çevirim (widening conversion)**
- Bit yapısı olarak geniş olandan dar olana: **Daraltan çevirim (narrowing conversion)**

```
boolean b = 12;      // İmkansız çevirim, int → boolean
long uzun = 5;       // Genişleten çevirim, int → long
int tamsayi = uzun;  // Daraltan çevirim, long → int
```

İmkansız Çevirim



- İmkansız çevirim, `boolean` ile başka hiçbir ilkel tip arasında çevirim olamayacağını söyler
- `byte` ve `short` ile `char` arasında da bir çevirim olamaz çünkü `byte` ve `short` büyüklük olarak `char` değerlerini ifade edemez
- Unutmayın `byte` ve `short` işaretli değerlerdir, `char` ise işaretsizdir
- Geri kalan durumlarda çevirim mümkündür, tam sayılar ve kayan noktalı sayılar birbirlerine çevirilebilirler



Genişleten Çevirimler

Genişleten Çevirimler - I



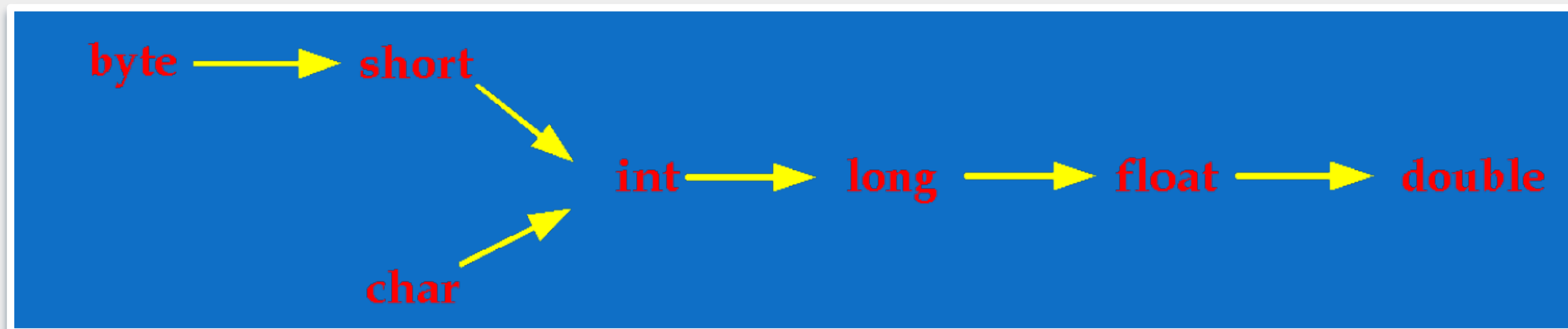
- **Genişleten çevirim (widening conversion)**, bit sunumu açısından daha dar olan bir tipten daha geniş olan bir tipe yapılan çevirimlerdir
- Java'da genişleten çevirimler otomatik olarak yapılır, çevirimin olması için atama yapmak dışında başka bir şeye gerek yoktur

```
long l = 5;    // Genişleten çevirim, int → long  
  
byte b = 12;   // 12, int olmasına rağmen derleyici onu 8 bite çevirir  
int j = b;     // Genişleten çevirim, byte → int  
  
float f = 9.74F;  
double d = f;  // Genişleten çevirim, float → double
```

Genişleten Çevirimler - II



- Genişleten çevirimler hiçbir zaman çalışma zamanı hatası vermezler
- Genişleten çevirimler şunlardır:



Genişleten Çevirimler - III



- Genişleten çevirimlerde bilgi kaybının olmayacağı gibi yanlış bir varsayım vardır
- Tam sayıların ve kayan noktalı sayıların kendi içlerindeki genişleten çevirimlerde bilgi kaybolmaz
- Örneğin `int`, `long`'a ya da `float`, `double`'a bilgi kaybı olmadan çevrilir çünkü bit sayısı açısından hedef tip, kaynak tipten daha geniştir

```
long l = 5;    // Genişleten çevirim, bilgi kaybı yok  
  
float f = 9.74F;  
double d = f; // Genişleten çevirim, bilgi kaybı yok
```

Genişleten Çevirimler - IV



- Ama tam sayılardan kayan noktalı sayılara yapılan çevirimlerde bilgi kaybı olabilir
- Örneğin `int`'ten `float`'a ya da `long`'tan `float` veya `double`'a yapılan çevirimlerde bilgi kaybı olabilir
- Çünkü `float` ve `double`, ondalıklı kısım için de yer ayırır
- Bilgi kaybı çevirilen sayıya bağlıdır, çevirilenin büyüklüğüne bağlı olarak, bitlerinin hepsi yeni tipe aktarılamayabilir

Genişleten Çevirimler - V



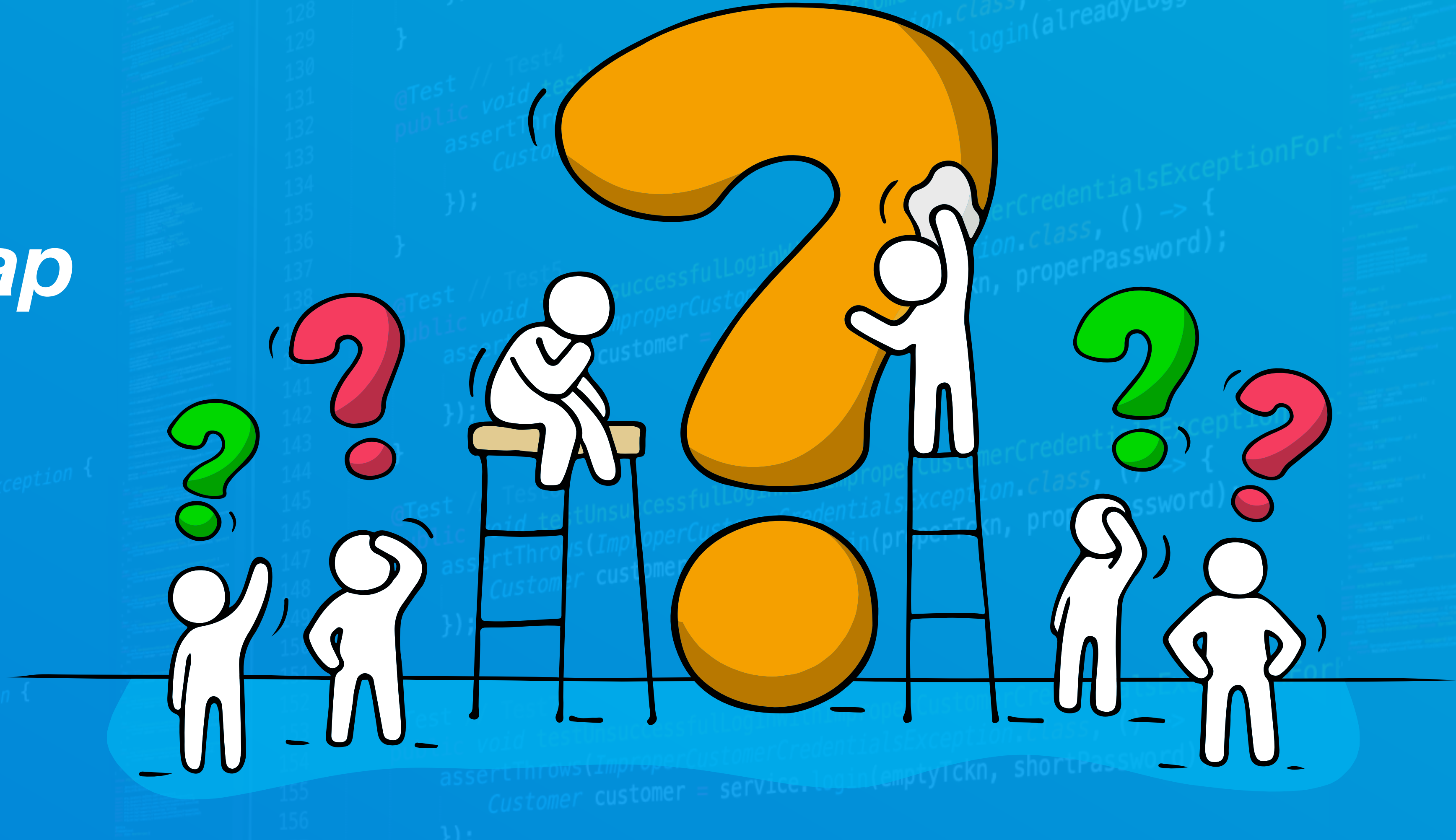
- Ayrıca tam sayılardan kayan noktalı sayılara yapılan çevirimlerde sayı artık kaynak noktalı hale geldiğinden virgülden sonra 0'lar eklenir

WideningConversion



- `WideningConversion`

Soru ve Cevap Zamanı!





Daraltan Çevirimler

Daraltan Çevirimler - I



- **Daraltan çevirim**, (**narrowing conversion**), bit sunumu açısından daha geniş olan bir tipten daha dar olan bir tipe yapılan çevirimlerdir
- Daraltan çevirimler şunlardır:

double → float → long → int → char → short → byte

- Java'da daraltan çevirimler otomatik olarak yapılmaz, derleyici hata verir

```
float f = 3.14; // Derleyici hatası, double → float  
int i = 5L;    // Derleyici hatası, long → int
```

Daraltan Çevirimler - II



- Daraltan çevirim yapabilmek için **çevirme işlemcisi (cast operator)** olan **()** kullanılır:
- Atama yapılırken, çevirilen tip, çevirme işlemcisi içine yazılır

```
float f = (float) 3.14; // Başarılı derleme  
int i = (int) 5L;      // Başarılı derleme
```

- Bu, Java'nın çevirimi otomatik olarak yapmayıp, sizden muhtemel bilgi kaybını göz önüne aldığınıza dair bir teyit/onay istemesidir

Daraltan Çevirimler - III



- Çevirme işlemcisi ile çalışma zamanında hata oluşmaz
 - Fakat çevirme sonucunda bir veri kaybı olabilir
- Çevirim sonucunda, çevirilen verinin büyüklüğüne bağlı olarak, geniş tipteki bitlerin hepsi yeni tipe aktarılamayabilir
- Bu da beklenmedik sonuçlara sebep olabilir

Daraltan Çevirimler - IV



- Kayan noktalı tiplerden tam sayı tiplere yapılan çevirimlerde, sayının virgülden sonraki kısmı atılır (truncate)
- Geniş tam sayı tipinden daha dar tam sayı tipine çevirimde, çevrilen değere bağlı olarak, garip sonuçlar çıkabilir
- Ayrıca negatif değer halinde `char` tipine çevirimde, daha önce bahsedilen kurala dikkat etmek gerekir
- Bu sebeple daraltan çevirimlerde dikkatli olunmalıdır

Daraltan Çevirimler - V



```
long lo1 = 5;
int i1 = (int) lo1;           // Bilgi kaybı yok.

long lo2 = 100_000_000_000L;
int i2 = (int) lo2;           // Bilgi kaybı var.

double d1 = 1_918.843_184_454_401;
float f1 = (float) d1;        // Bilgi kaybı var.

double d2 = 3.14;
float f2 = (float) d2;        // Bilgi kaybı yok

double d3 = 3.14;
int i3 = (int) d3;             // 0.14 kaybolur.

char char1 = (char) -65;       // Value is converted to a positive
                                // number by adding 65536
```

NarrowingConversion



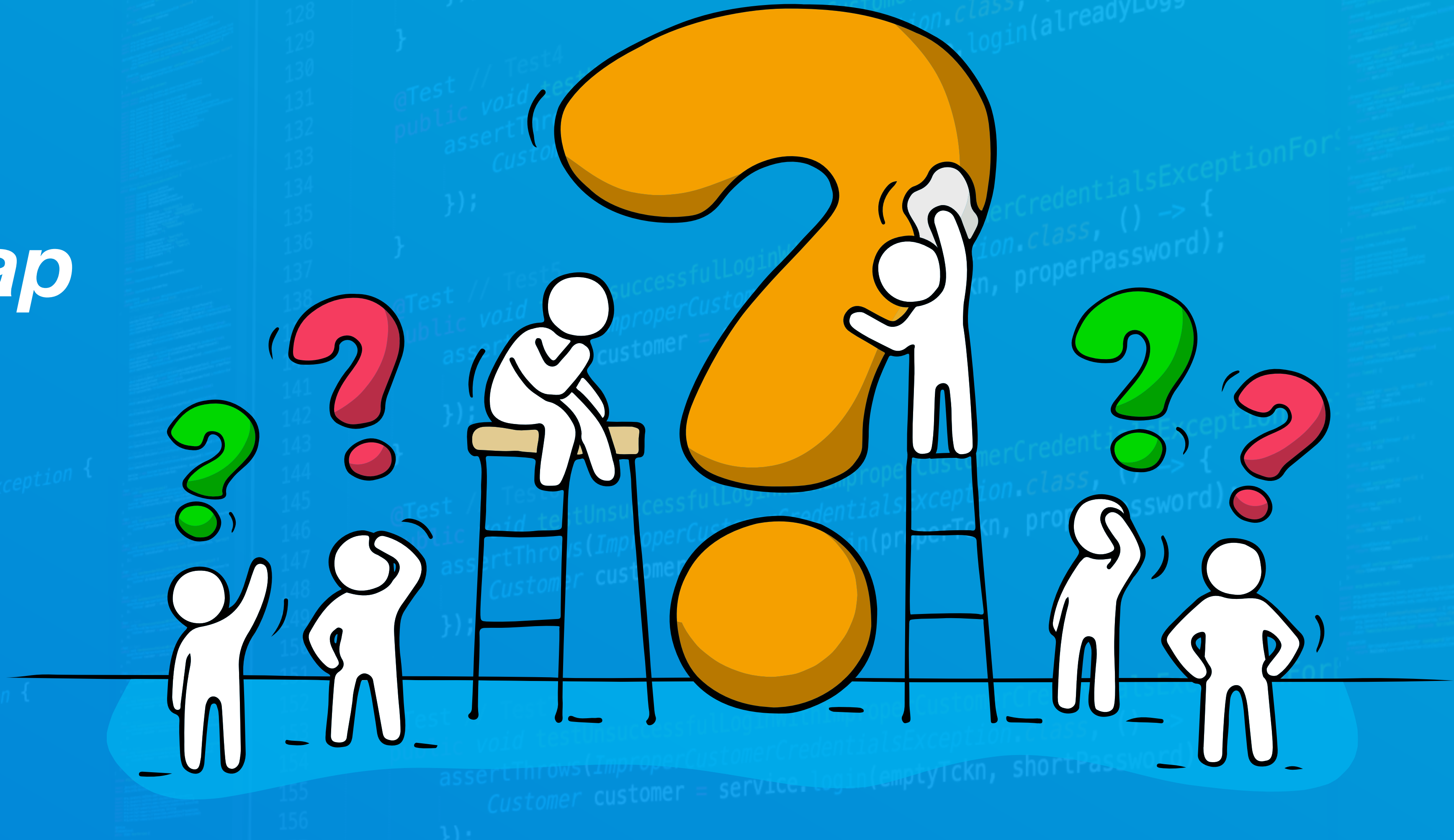
- `NarrowingConversion`

PowerDemo



- `conversion.PowerDemo`

Soru ve Cevap Zamanı!



Ödevler





1. `main` metodu olan `Variables` isimli bir sınıf oluşturun
 - a. `main` metot içinde 8 farklı tipte ilkel değişkeni, ilk değer atamadan oluşturun ve anlamlı ve kod geleneğine uygun isimler verin
 - b. `main` metot içinde yerel değişkenleri basın, derleyicinin hatasını yorumlayıp gerekli düzeltmeleri yapın



2. `Casts` isimli bir sınıf oluşturup, içine koyacağınız `main` metot içinde, farklı ilkel tiplerde değişkenler tanımlayarak daraltan ve genişleten çevirimler yapın
 - a. Daraltan çevirimleri cast operatörü olmadan yaptığınızda aldığınızı hatayı yorumlayın
 - b. Çevirimlerden önce ve sonra değişkenleri basarak, değerlerindeki değişimleri gözlemleyin



3. Bir çalışanın ayda 45.912,75 TL kazandığını düşünün. Sadece ilkel tipler arasındaki çevirimleri kullanarak bu maaşın tam ve kuruşlu kısmını nasıl elde edersiniz?



4. Aşağıdaki ifadelerinin kaç basacağını tahmin edin ve sonra kodu yazarak test edin

a. Test için `jshe11`'i kullanabilirsiniz

```
System.out.println((double) 4);  
System.out.println((int) 10.0);  
System.out.println((char) 104 / 2);  
System.out.println((char) (104 / 2));
```



5. Yandaki kod parçalarında değer üreten ifadeler ve sonrasında üretilen değeri basan satırlar var
- Bu kod parçalarında, özellikle ifadelerde problem olup olmadığını belirleyin; problem varsa düzeltin ve ne basacağını tahmin edin
 - Sonra bu kodları yazıp sonucu gözlemleyin
 - Test için `jshe11`'i kullanabilirsiniz

```
int i = 10;
double d = a;
System.out.println(d);

double d = 10.8;
int i = (int) c;
System.out.println(i);

byte b1 = 50;
byte b2 = 60;
byte b3 = b1 + b2;
System.out.println(b3);

System.out.println((char) 65);
System.out.println((int) '9');
```

Bundan Sonra?



Bundan Sonra?



- Şimdi de karmaşık tip ve değişkenleri inceleyeceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

